

MULTI-THREAD PROGRAMMING

CONTENTS OF THE CHAPTER

Chapter 8 addresses issues related to multithreading programming, methods for creating threads, and importantly, handling synchronization-related problems .

- *How to create threads using Thread and Runnable.*
- *Synchronization processes using locks.*
- *Use Fork-Join.*
- *Create and manage processes.*

8.1. SOME BASIC CONCEPTS

8.1.1. Polling loop and Interrupt

The CPU fetches instructions from memory hierarchy (mainly from CPU cache and ultimately from RAM) and executes them.

The CPU and main memory are just two of many components in a real computer system. A computer system includes other devices such as: hard drives, keyboards, mice, monitors, network devices, etc. The list of devices is not fixed, and computer systems can be expanded by adding new devices.. The CPU must somehow communicate with and control all these devices. The way it works is that each device in the system has a device driver. This driver acts as an intermediary, allowing the CPU to communicate with the various devices. Therefore, installing a new device on a system usually involves two steps: plugging the physical device into the computer and installing the device driver software. Without a device driver, the physical device would be useless, as the CPU would be unable to communicate with it.

A computer system comprising multiple devices is typically organized by connecting those devices to one or more buses. A bus is a set of wires carrying various types of information between the devices connected to it. These wires transmit data, addresses, and control signals. Addresses direct

data to a specific device or a component of that device.

A simple computer system can be organized as follows:

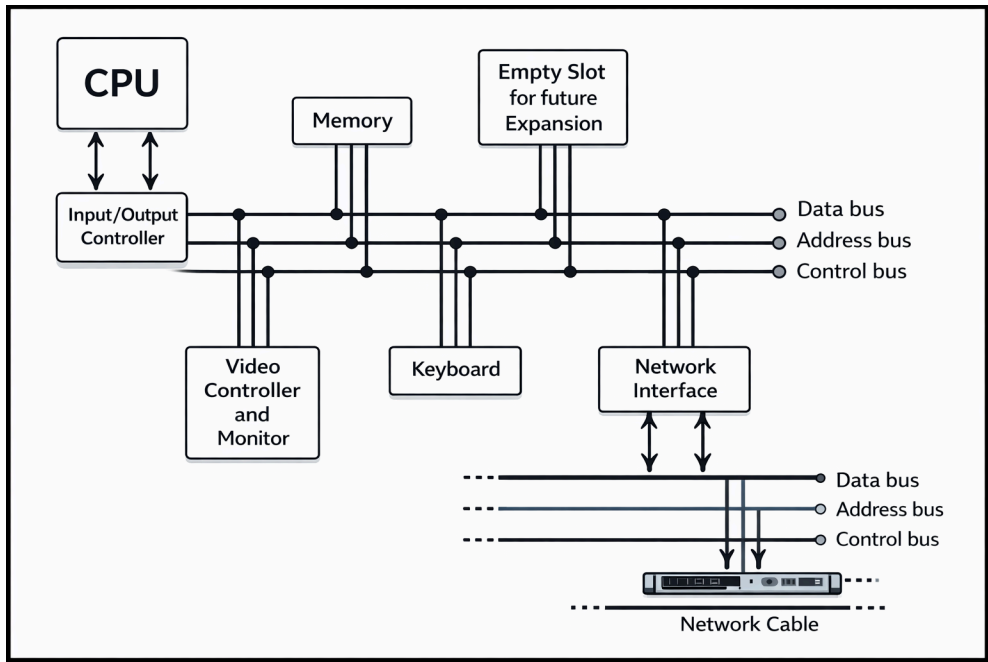


Figure 8-1. Organization of a simple computer system

Devices such as keyboards, mice, and network interfaces interact with the CPU through interrupt mechanisms.

How does the CPU know that the data is there? A simple but inefficient approach is to let the CPU continuously check whether input devices have data available. Whenever it finds the data, it processes it. This method is called polling, because the CPU continuously polls input devices to see if they have any input data to report. While polling is simple, it is also very inefficient. The CPU would waste a lot of time just waiting for input.

To avoid this inefficiency, interrupts are often used instead of polling. An interrupt is a signal sent by another device to the CPU. When an interrupt occurs, the CPU temporarily suspends the execution of the current instruction sequence and transfers control to an interrupt handler to process the interrupt. Once it has processed the interrupt, it returns to what it was doing before the interrupt occurred. For example, when a key is pressed on a

computer keyboard, a keyboard interrupt is sent to the CPU. The CPU responds to this signal by interrupting what it is doing, reading the pressed key, processing it, and then returning to the task it was performing before the key was pressed. Interrupts help the CPU work efficiently with "asynchronous" processing, that is, at unpredictable times.

Another example of interrupt usage: consider what happens when the CPU needs to access data stored on a hard drive. The CPU can only access data directly if it's in main memory. Data on the disk must be copied to memory before it can be accessed. However, hard drives are extremely slow compared to main memory. When the CPU needs data from the disk, it sends a signal to the drive telling it to locate the data and get ready (this signal is sent synchronously, under the control of a regular program). Then, instead of just waiting for the long and unpredictable amount of time the drive would take to do this, the CPU proceeds with some other task. When the drive has the data available, it sends an interrupt signal to the CPU. The interrupt handler can then read the requested data.

All modern computers use multitasking to perform multiple tasks simultaneously. Some computers can be used by multiple people at the same time. The CPU is so fast that it can quickly switch services from one user to another, dedicating a fraction of a second to each user. This multitasking application is called time-sharing. But a modern personal computer with only a single user also uses multitasking.

8.1.2. Synchronous and Asynchronous Processing

Modern computers support concurrent execution of multiple threads. When multiple CPU cores are available, threads may also execute in parallel. This means that many tasks can be handled concurrently. Programming languages can implement this through multithreading, synchronous and asynchronous processing:

- 1) **Synchronous processing** means that the code will run sequentially in the order it's written – that is, the code below must wait until the code above returns a result.
- 2) **Asynchronous processing** refers to processes where the code

below can continue to execute even though the code above has not yet finished processing and returned a result.

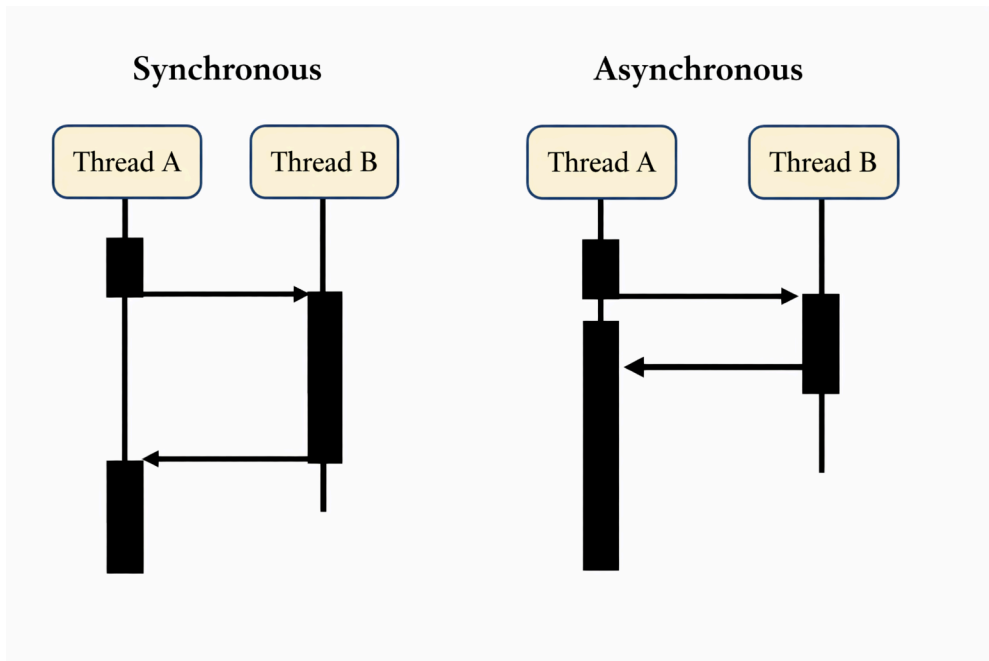


Figure 8-2. Synchronous and asynchronous processing

Many CPUs can execute more than one job simultaneously; such CPUs contain multiple cores, each of which can run one or more threads. Because multiple threads are frequently executed concurrently, computers must have mechanisms to allocate time between threads, much like how a computer shares its service time between users. Generally, a sequence of instructions being executed will continue running until one of the following occurs:

- A thread can automatically relinquish control, giving other threads a chance to run.
- A thread may need to wait for asynchronous events to occur. For example, a command sequence might request some data from the disk or wait for the user to press a key. While waiting, the thread is supposedly blocked, and other threads (if any) will have a chance to run. When the event occurs, an interrupt will “wake up” the thread so it can continue running.

A thread can use up its allocated time and be suspended to allow other threads to run. Most computers can “force” a thread to suspend in this way;

8.2. PROCESSES AND THREAD

- A program that is being executed is usually called a process. A thread is the smallest unit of execution within a process.
- A process may contain one or more threads. A thread is the smallest part of a process that can execute concurrently.
- A process has its own address space. Threads share the address space of **the same process**.
- A process can communicate with another process using IPC. A thread can communicate directly with another thread.
- Creating a new thread is generally lighter and faster than creating a new process, because processes require separate memory spaces and additional system resources.
- A process can control other processes. A process cannot control its sibling processes, but can only control its child processes.

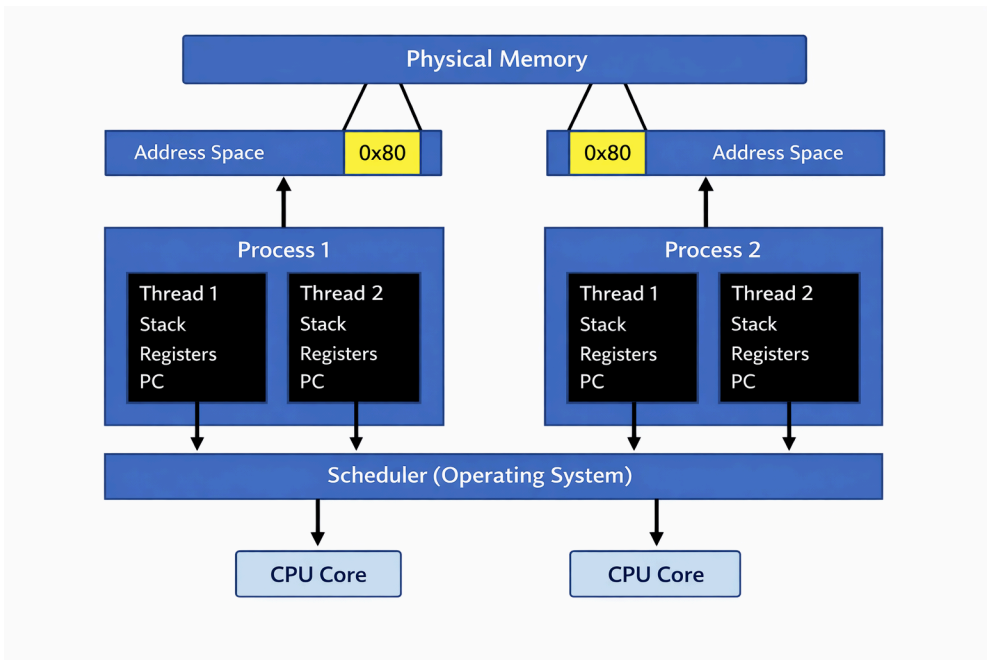


Figure 8-3. How processes and threads work.

8.3. THREAD IN JAVA

8.3.1. Time-sharing mechanism

Java has built-in support for concurrent programming by running multiple threads simultaneously within a single program. A thread is a lightweight unit of execution within a process, is a single stream of sequential programming activity, with a defined beginning and end. Throughout the thread's lifespan, there is only one execution point. A thread itself is not a program because it cannot run on its own. Instead, it runs within a program. The following figure shows a program with three threads running under a single CPU:

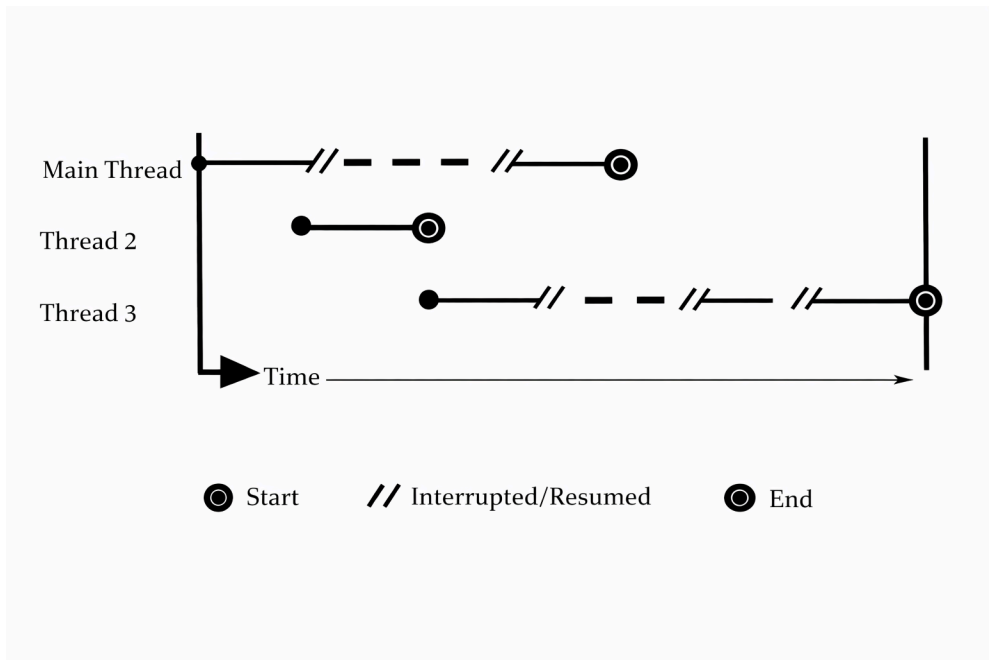


Figure 8-4. Time-sharing mechanism

8.3.2. How to create threads in Java

There are two main steps to creating a thread:

Step 1 : Create a Runnable object (using the Thread Class or Runnable Interface).

Step 2 : Create a Thread and start it.

For example: Create a "RunThread" class that inherits from the "Thread" class.

```
public class RunThread extends Thread {
    private String toSay ;
    private int Sleep ;

    public RunThread ( String st, int sl) {
        toSay = st;
        Sleep = sl;
    }
}
```

```

public void run () {
    try {
        for (; ; ) {
            System . out .println( toSay );
            Thread.sleep ( Sleep ) ;
        }
    } catch ( InterruptedException e) {
    }
}

```

Write a main() method in a class to display the result:

```

public static void main ( String [] args) {
    RunThread thr1 = new RunThread( "Thread A" , 1200 );
    RunThread thr2 = new RunThread( "Thread B" , 2000 );
    thr1 .start();
    thr2 .start();
}

```

Run a test and check the displayed results:

```

Thread A
Thread B
Thread A
Thread B
Thread A
Thread A
Thread B
Thread A
...

```

8.4. RUNNABLE INTERFACE

Modify the RunThread Implements Runnable Interface class.

The code inside the RunThread remains the same.

```
public class RunThread implements Runnable {  
    ...  
}
```

Revise the main method as follows:

```
public static void main ( String [] args) {  
    RunThread out1 = new RunThread( "Thread A" , 2000 );  
    RunThread out2 = new RunThread( "Thread B" , 1200 );  
    Thread thr1 = new Thread( out1 );  
    Thread thr2 = new Thread( out2 );  
    thr1 .start();  
    thr2 .start();  
}
```

Run a test and check the displayed results:

```
Thread A  
Thread B  
Thread A  
Thread B  
Thread A  
Thread A  
Thread B  
Thread A  
...
```

8.5. ANONYMOUS THREAD

```
public static void main(String[] args) {  
    new Thread(() -> {  
        System.out.println ( "Start" ) ;  
    }) .start();  
}
```

```
System.out.println ( " Execute the job " );  
System.out.println ( "End" ) ;  
}).start();  
}
```

8.6. STATE OF THE THREAD

Seven states of a thread. A generated thread can exist in one of the following 6 states:

The lifecycle of a thread is shown in the following diagram:

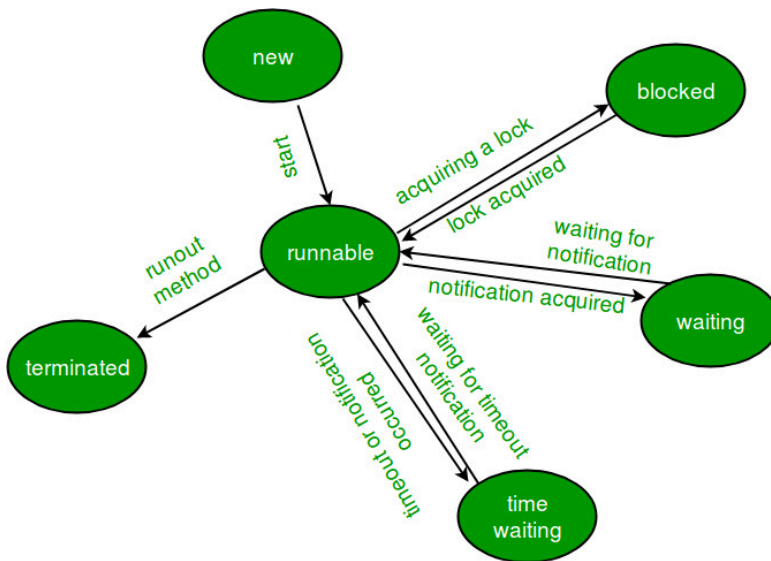


Figure 8-5. Lifecycle of a thread

Some methods:

- **setName()** : Sets the name for the Thread.
- **getName()**: Returns the Thread name.
- **isAlive()** : Checks if the thread is still running.
- **join()**: Waits until a thread finishes.
- **wait()** : Waits until a Thread releases the resource.
- **notify()** : Notifies the waiting thread.

8.7. SYNCHRONIZATION PROCESSING

The **synchronized** keyword ensures mutual exclusion and thread-safe access to shared data. The **volatile** keyword ensures that changes to a variable are immediately visible to other threads by preventing caching inconsistencies.

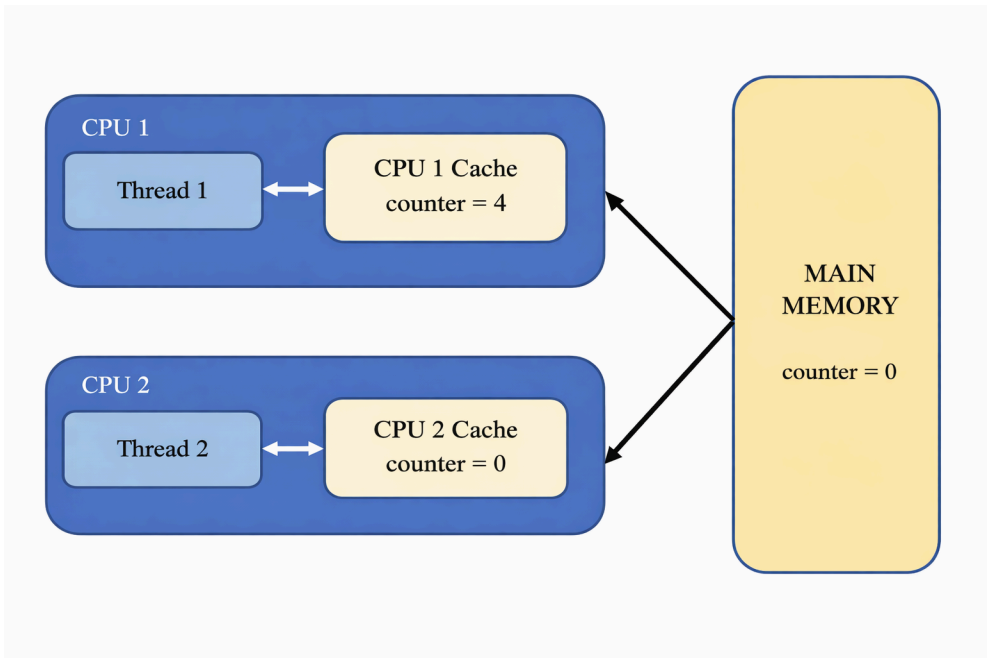


Figure 8-6. Data from variables can be loaded from the CPU's cache.

8.8. USING FORK-JOIN

The fork/join framework provides tools to help accelerate parallel processing by attempting to maximize CPU performance, working on a divide-and-conquer principle:

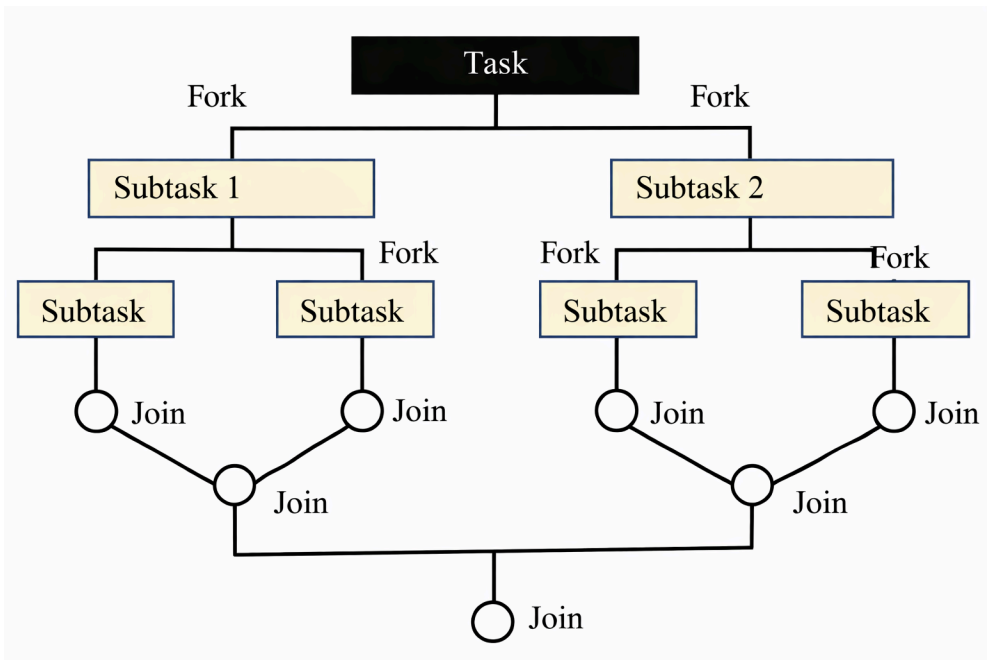


Figure 8-7. Fork/join framework

8.9. PROCESS

Java can create new processes using the `Process` and `ProcessBuilder` classes. Creating a process can start any application. For example, the following code creates a process and runs a command to browse the `/bin/ls` directory.

```
public static void main(String[] args) throws IOException,
InterruptedException {
    Process p = new ProcessBuilder( "/bin/ls" , "-l" )
        .directory(Paths.get ( "/"tmp" ).toFile())
        .start();
    try ( var in = new Scanner(p.getInputStream())) {
        while (in.hasNextLine())
            System.out.println(in.nextLine());
    }
}
```

PRACTICE

Lesson 1. Join Thread

Create class “MyThread” that implements Runnable Interface.

Program for the "MyThread" class.

```
class MyThread implements Runnable {
    String name ; // name of thread
    Thread t ;

    MyThread ( String threadname) {
        name = threadname;
        t = new Thread( this , name );
        System . out .println( "New thread: " + t );
        t.start ();
    }

    public void run () {
        try {
            for ( int i = 5 ; i > 0 ; i--) {
                System . out .println( name + ": " + i);
                Thread.sleep ( 1000 ) ;
            }
        } catch ( InterruptedException e) {
            System . out .println( name + " interrupted." );
        }

        System . out .println( name + " exiting." );
    }
}
```

Write a main() method in a class to create and start three threads:

```
public static void main ( String [] args) {
    MyThread ob1 = new MyThread( "One" );
    MyThread ob2 = new MyThread( "Two" );
    MyThread ob3 = new MyThread( "Three" );
}
```

```
System . out .println( "Thread One is alive: " + ob1 . t
.isAlive());
System . out .println( "Thread Two is alive: " + ob2 . t
.isAlive());
System . out .println( "Thread Three is alive: " + ob3 . t
.isAlive());

System . out .println( "Main thread exiting." );
}
```

Run a test and check the results:

```
New thread: Thread[One,5,main]
New thread: Thread[Two,5,main]
New thread: Thread[Three,5,main]
Thread One is alive: true
Thread Two is alive: true
Thread Three is alive: true
Thread One is alive: true
Thread Two is alive: true
Thread Three is alive: true
Main thread exiting.
Three: 5
One: 5
Two: 5
Three: 4
Two: 4
One: 4
One: 3
Three: 3
Two: 3
One: 2
```

```
Three: 2
Two: 2
Two: 1
One: 1
Three: 1
One exiting.
Two exits.
Three exiting.
```

Modify the main method as follows, adding join():

```
public static void main ( String [] args) {
    MyThread ob1 = new MyThread( "One" );
    MyThread ob2 = new MyThread( "Two" );
    MyThread ob3 = new MyThread( "Three" );

    System . out .println( "Thread One is alive: " + ob1 . t
.isAlive());
    System . out .println( "Thread Two is alive: " + ob2 . t
.isAlive());
    System . out .println( "Thread Three is alive: " + ob3 . t
.isAlive());

    try {
        System . out .println( "Waiting for threads to finish."
);
        ob1.t.join ( ) ;
        ob2.t.join ( ) ;
        ob3.t.join ( ) ;
    } catch ( InterruptedException e) {
        System . out .println( "Main thread Interrupted" );
    }

    System . out .println( "Thread One is alive: " + ob1 . t
.isAlive());
}
```

```

    System . out .println( "Thread Two is alive: " + ob2 . t
.isAlive());

    System . out .println( "Thread Three is alive: " + ob3 . t
.isAlive());

    System . out .println( "Main thread exiting." );
}

```

Run the test and check the results (*). Explain the printed results.

Lesson 2. The Prime Number Problem (*)

This exercise will involve writing a program using multiple threads to count prime numbers from 0 to 1,000,000. The number of threads required to search for prime numbers will be determined by the user.

Create a "CountPrimesThread" class that extends the "Thread" class.

Declare the variables:

```

//Variable to store the number of primes in a specified range
of
static int count = 0 ;
// Variable to store the range of minimum and maximum integer
value
int min , max ;

```

Write the constructors:

```

public CountPrimesThread ( int min, int max) {
    this.min = min ;
    this.max = max ;
}

```

The `isPrime()` method is called to check if x is a prime number.

```

private static boolean isPrime ( int x) {
    int top = ( int ) Math . sqrt (x);
    for ( int i = 2 ; i <= top ; i++)
        if (x % i == 0 )
            return false ;
}

```

```
        return true ;  
    }  
}
```

The `countPrimes()` method is called to count the prime numbers between min and max:

```
private static void countPrimes ( int min, int max) {  
    for ( int i = min; i <= max; i++)  
        if ( isPrime (i))  
            count ++;  
}
```

The `Run()` method is called to output a message about the number of prime numbers it found:

```
public void run () {  
    countPrimes ( min , max );  
    System . out .println( "There are " + count + " primes  
between " + "" + min + " and " + max );  
}
```

Write a method called `CountPrimesWithThreads()` to create a thread that counts prime numbers:

```
public static void countPrimesWithThreads ( int  
numberOfThreads) {  
    int start = 0 ;  
    int to = 1000000 ;  
    int increment = to / numberOfThreads;  
    System . out .println( " \n Counting primes between " + (  
start + 1 ) + " and " + to + " using " + numberOfThreads + "  
threads. " + "Please wait... \n " );  
    long startTime = System . currentTimeMillis ();  
    CountPrimesThread [] worker = new  
CountPrimesThread[numberOfThreads];  
  
    for ( int i = 0 ; i < numberOfThreads; i++) worker [i] = new  
CountPrimesThread( start + i * increment + 1 , start + (i + 1 )  
* increment );  
}
```

```

    for ( int i = 0 ; i < numberOfThreads; i++) worker
[i].start();

    for ( int i = 0 ; i < numberOfThreads; i++) {
        while ( worker [i].isAlive()) {
            try {
                worker [i].join();
            } catch ( InterruptedException e) {
            }
        }
    }

    long elapsedTime = System . currentTimeMillis () -
startTime ;

    System . out .println( " \n Total elapsed time: " + (
elapsedTime / 1000.0 ) + "seconds. \n " );
}

```

Write a program for the main() method:

```

public static void main ( String [] args) {
    int numberOfThreads = 0 ;
    do {
        System . out .println( "How many threads do you want to
use (from 1 " + "to 10)?" );
        Scanner input = new Scanner( System . in );
        numberOfThreads = input .nextInt();
        if (numberOfThreads < 1 || numberOfThreads > 10 )
            System . out .println( "Please enter 1 .. 10!" );
    }
    while (numberOfThreads < 1 || numberOfThreads > 10 );

    countPrimesWithThreads(numberOfThreads);
}

```

Execute the program and test how the computation speed changes with different numbers of threads. Indicate when the computation time improves with increasing the number of threads and when the computation time does not improve.

Lesson 3. The Banking Problem

Create a Bank class:

```
import java.util.Arrays;

public class Bank {
    public final double [] accounts ;
    public Bank( int n, double initialBalance) {
        accounts = new double [n];
        Arrays. fill ( accounts , initialBalance);
    }

    public void transfer( int from, int to, double amount) {
        System. out .print(Thread. currentThread ().getName());
        accounts [from] -= amount;
        System. out .printf( " %10.2f from %d to %d" , amount, from,
to);
        accounts [to] += amount;
        System. out .printf( " Total Balance: %10.2f%n" ,
getTotalBalance());
    }

    public double getTotalBalance() {
        double sum = 0 ;
        for ( double a : accounts )
sum += a;
        return sum;
    }
}
```

Create the Main class:

```
public class Main {

    public static final int NACCOUNTS = 3 ;
    public static final double INITIAL_BALANCE = 1000 ;
    public static final double MAX_AMOUNT = 1000 ;
```

```

    public static void main(String[] args) throws
InterruptedException {
Bank bank = new Bank( NACCOUNTS , INITIAL_BALANCE );
    while ( true ) {
        int fromAccount = ( int ) (bank. accounts . length
* Math. random ());
        int toAccount = ( int ) (bank. accounts . length *
Math. random ());
        double amount = MAX_AMOUNT * Math. random ( );
bank.transfer(fromAccount, toAccount, amount);
Thread.sleep ( 10 ) ;
    }
}
}
}

```

Run a test and see the results. For example, if you want to receive input from the keyboard, it's not possible:

```

main 407.68 from 0 to 0 Total Balance: 3000.00
main 507.47 from 0 to 0 Total Balance: 3000.00
main 78.25 from 0 to 1 Total Balance: 3000.00
main 165.24 from 0 to 1 Total Balance: 3000.00
main 356.55 from 1 to 0 Total Balance: 3000.00
main 59.27 from 2 to 2 Total Balance: 3000.00
main 626.90 from 0 to 0 Total Balance: 3000.00
main 758.75 from 0 to 0 Total Balance: 3000.00

```

Now let's modify the above problem using Threads. First, create a class called ThreadTest and put the transaction into a thread:

```

public class ThreadTest {
    public static final int NACCOUNTS = 3 ;
    public static final double INITIAL_BALANCE = 1000 ;
    public static final double MAX_AMOUNT = 1000 ;

    public static void main(String[] args) {
Bank bank = new Bank( NACCOUNTS , INITIAL_BALANCE );
Runnable runnable = () -> {
    while ( true ) {

```

```

        int fromAccount = ( int ) ( bank . accounts .
length * Math. random ());
        int toAccount = ( int ) ( bank . accounts .
length * Math. random ());
        double amount = MAX_AMOUNT * Math. random ();
        bank .transfer(fromAccount, toAccount, amount);
    }
};

    new Thread(runnable).start();
}
}

```

Run a test and see the results:

```

Thread-0 251.53 from 1 to 0 Total Balance: 3000.00
Thread-0 642.04 from 0 to 1 Total Balance: 3000.00
Thread-0 308.59 from 1 to 1 Total Balance: 3000.00
Thread-0 484.27 from 2 to 1 Total Balance: 3000.00
Thread-0 462.66 from 2 to 0 Total Balance: 3000.00
Thread-0 263.15 from 1 to 0 Total Balance: 3000.00
Thread-0 904.53 from 1 to 1 Total Balance: 3000.00
Thread-0 70.65 from 2 to 0 Total Balance: 3000.00

```

Make all threads run:

```

public static void main(String[] args) {
    Bank bank = new Bank( NACCOUNTS , INITIAL_BALANCE );
    for ( int i = 0 ; i < NACCOUNTS ; i++) {
        int fromAccount = i;
        Runnable runnable = () -> {
            while ( true ) {
                int toAccount = ( int ) ( bank . accounts .
length * Math. random ());
                double amount = MAX_AMOUNT * Math. random ();
                bank .transfer( fromAccount , toAccount,
amount);
            }
        };
    }
};

```

```
new Thread(runnable).start();  
}  
}
```

Run a test and see the results:

```
Thread-0 263.14 from 1 to 2 Total Balance: 2356.72  
Thread-0 640.00 from 2 to 1 Total Balance: 2543.76  
Thread-2 454.85 from 2 to 2 Total Balance: 2543.76  
Thread-2 452.96 from 1 to 0 Total Balance: 2963.40  
Thread-0 443.90 from 1 to 1 Total Balance: 2963.40  
Thread-0 411.98 from 1 to 2 Total Balance: 2963.40  
Thread-0 272.90 from 0 to 0 Total Balance: 2963.40  
Thread-0 3.28 from 1 to 1 Total Balance: 2752.24  
Thread-1 606.72 from 0 to 0 Total Balance: 2752.24  
Thread-1 129.45 from 0 to 0 Total Balance: 2752.24  
Thread-1 173.33 from 1 to 2 Total Balance: 2752.24  
Thread-1 214.44 from 0 to 2 Total Balance: 2668.70  
Thread-0 726.02 from 2 to 0 Total Balance: 2668.70  
Thread-0 701.75 from 0 to 1 Total Balance: 2668.70
```

The total amount is incorrect because it was subtracted and printed but not added to the account below.

Use `SyncThread`.

Revise the `Bank` class to add a lock attribute:

```
private Lock lock ;
```

In `Bank`'s constructor, initialize a `ReentrantLock`:

```
public Bank( int n, double initialBalance) {  
    accounts = new double [n];  
    Arrays.fill ( accounts , initialBalance);  
    lock = new ReentrantLock();  
}
```

When transferring, lock and unlock after the transfer is complete.

```

public void transfer( int from, int to, double amount) {
    lock .lock();
    System. out .print(Thread. currentThread ().getName());
    accounts [from] -= amount;
    System. out .printf( " %10.2f from %d to %d" , amount, from,
to);
    accounts [to] += amount;
    System. out .printf( " Total Balance: %10.2f%n" ,
getTotalBalance());
    lock.unlock ();
}

```

Use `try finally` to ensure the lock is unlocked:

```

public void transfer( int from, int to, double amount) {
    lock .lock();
    try {
        System. out .print(Thread. currentThread ().getName());
        accounts [from] -= amount;
        System. out .printf( " %10.2f from %d to %d" , amount, from,
to);
        accounts [to] += amount;
        System. out .printf( " Total Balance: %10.2f%n" ,
getTotalBalance());
    } finally {
        lock.unlock ();
    }
}

```

To resolve the conflict issue, the program needs to be modified to work synchronously. First, add a conditional object to the `bank` class:

```

private Condition condition;

```

Initialize the conditional object in the Bank creation method:

```

public Bank( int n, double initialBalance) {
    accounts = new double [n];
    Arrays. fill ( accounts , initialBalance);
    lock = new ReentrantLock();
}

```

```
condition = lock .newCondition(); //Additional
}
```

In the transaction (transfer function), instead of checking:

```
if ( accounts [from] < amount) return ;
```

By checking as follows:

```
while ( accounts [from] < amount) {
    try {
System. out .println(Thread. currentThread ().getName() + " "
+ from + " is not enough" );
        condition.await ();
    } catch (InterruptedException e) {
e.printStackTrace();
    }
}
```

After the transaction is complete, call the signalAll() method:

```
condition.signalAll ();
```

The transfer method would then be as follows:

```
public void transfer( int from, int to, double amount) {
    lock .lock();
    try {
        while ( accounts [from] < amount) {
            try {
System. out .println(Thread. currentThread ().getName() + " "
+ from + " is not enough" );
                condition.await ();
            } catch (InterruptedException e) {
e.printStackTrace();
            }
        }
        System. out .print(Thread. currentThread ().getName());
        accounts [from] -= amount;
        System. out .printf( " %10.2f from %d to %d" , amount, from,
to);
    }
}
```

```

        accounts [to] += amount;
System. out .printf( " Total Balance: %10.2f%n" ,
getTotalBalance());
        condition.signalAll ();
finally {
        lock.unlock ();
}
}

```

The second way to solve the synchronization problem is to use **synchronized** along with **wait** and **notifyAll**.

Remove the locks and conditions, and use synchronized wait notifyAll in the transfer method.

```

public synchronized void transfer( int from, int to, double
amount) {
    while ( accounts [from] < amount) {
        try {
System. out .println(Thread. currentThread ().getName() + " "
+ from + " is not enough" );
wait();
        } catch (InterruptedException e) {
e.printStackTrace();
        }
    }
System. out .print(Thread. currentThread ().getName());
    accounts [from] -= amount;
System. out .printf( " %10.2f from %d to %d" , amount, from,
to);
    accounts [to] += amount;
System. out .printf( " Total Balance: %10.2f%n" ,
getTotalBalance());
notifyAll();
}

```

Change MAX_AMOUNT to 1500.

Rerunning the program will reveal the deadlock that occurred:

```
Thread-0 0 is not enough
Thread-2 is not enough
Thread-1 907.57 from 1 to 2 Total Balance: 3000.00
Thread-0 0 is not enough
Thread-1 1 is not enough
Thread-2 1078.15 from 2 to 1 Total Balance: 3000.00
Thread-2 is not enough
Thread-0 0 is not enough
Thread-1 142.17 from 1 to 0 Total Balance: 3000.00
Thread-1 120.96 from 1 to 2 Total Balance: 3000.00
Thread-1 1 is not enough
Thread-2 is not enough
Thread-0 0 is not enough
```

Try to think of a way to resolve the deadlock that occurred in the above scenario.

Lesson 4. Using Thread Pools (*)

Create a thread to simulate a file download:

```
public class DownloadFile implements Runnable {
    @Override
    public void run() {
        int timetoFinish = ( int ) ( 5000 * Math. random ());
        System.out.println ( Thread.currentThread () . getName() +
            "Starting to load a new file. Requires" + timetoFinish + "ms
            to complete" );
        try {
            Thread.sleep ( timetoFinish );
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println ( Thread.currentThread () . getName() +
            "Download Completed" );
    }
}
```

```
}  
}
```

Create a Thread Pool by writing the following in the main method:

```
public static void main(String[] args) {  
    ExecutorService executor = Executors. newFixedThreadPool ( 5  
);  
    for ( int i = 0 ; i < 10 ; i++) {  
        Runnable worker = new DownloadFile();  
        executor.execute(worker);  
    }  
}
```

Run a test and explain how the program works.

Lesson 5. Practice Using the fork-join Framework

```
public class ForkJoinTest {  
    public static void main(String[] args) {  
        final int SIZE = 100000000 ; //100M  
        var numbers = new double [SIZE];  
        for ( int i = 0 ; i < SIZE; i++)  
            numbers[i] = Math.random ( );  
        var counter = new Counter(numbers, 0 , numbers. length  
);  
        var pool = new ForkJoinPool();  
        pool.invoke(counter);  
        System. out .println(counter.join());  
    }  
}  
  
class Counter extends RecursiveTask<Integer> {  
    private double [] values ;  
    private int from ;  
    private int to ;  
  
    public Counter( double [] values, int from, int to) {  
        this.values = values ;  
    }  
}
```

```

        this.from = from ;
        this . to = to;
    }

    protected Integer compute() {
        if ( to - from < 1000 ) {
            int count = 0 ;
            for ( int i = from ; i < to ; i++)
                if ( values [i]> 0.5 ) count++;
            return count;
        }
        else {
            int mid = ( from + to ) / 2 ;
            var first = new Counter( values , from , mid);
            var second = new Counter( values , mid, to );
            invokeAll (first, second);
            return first.join() + second.join();
        }
    }
}

```

QUIZZES

What is the difference between a thread and a process?	
A thread occupies a dedicated memory space.	
Threads share memory space.	
A thread contains multiple processes inside it.	
The thread runs slower than the process.	

The thread is created by:	
Declare the class implementation from the Runnable interface.	
Declare a class that inherits from the Thread class.	
Use the method of creating anonymous threads.	
All of the above answers are correct.	

A thread is in the running state when:	
--	--

It is running and being served by the CPU	
It is waiting for another thread.	
It is locked or waiting for an input/output operation.	
All of the above answers are correct.	

The join method is used to:	
Announcement: Processes are temporarily paused, awaiting further processing and resumption.	
Check if a process is alive.	
Wait for another thread to release the resource.	
Wait for another process to finish.	

The keyword 'synchronized' is used for:	
Solving the asynchronous problem	
Synchronize data access	
Ensure a single thread executes a block of code.	
Resolving deadlock issues	

To start a thread, use the following method:	
run()	
action()	
ready()	
start()	

One of the purposes of multithreaded programming is:	
Increase computing speed	
Maximizing the use of computer resources.	
Multitasking	
All the answers are correct.	

The disadvantages of multithreading are:	
Complex processing	

Speed is reduced	
Work was disrupted due to the use of time-sharing.	
All the answers are wrong.	

To create a thread, when should you implement from the Runnable interface instead of inheriting from the Thread class?	
When you want to write shorter code	
When you want to create more threads	
When a class is running, it still wants to inherit from another class.	
When you want the code to be clear and explicit.	

A thread will transition from the new state to the running state when:	
The operating system allows threads to operate.	
Call the start() function.	
Call the run() function.	
Immediately after the thread is created	

Please indicate the number of threads that will change and print the value of the variable i: <pre> class CountAndPrint implements Runnable { private final String name ; CountAndPrint (String name) { this.name = name ; } static int i = 0 ; @Override public void run () { while (i < 10) System . out .println(this . name + ": " + i ++); } </pre>	
---	--

<pre>public static void main (String [] args) { for (int t = 1 ; t <= 4 ; t ++) new Thread(new CountAndPrint("Instance " + t)).start(); while (i < 10) System . out .println("Main: " + i ++); } }</pre>	
4	
5	
9	
10	

PRACTICE EXERCISES

Exercise 1. Calculate the sum of the elements in an array.

Write a Java program using multithreading to calculate the sum of the elements in an integer array. The integer array has more than 1000 elements.

Exercise 2. Calculate the sum of the elements in an array.

Write a Java program that creates two streams: one that prints even numbers and one that prints odd numbers. The stream printing even numbers must start before the stream printing odd numbers. After printing, calculate the sum of the elements and print it to the screen.

Exercise 3. Calculate the sum of an array of integers using ForkJoinPool

Write a program to calculate the sum of an array of integers using ForkJoinPool. Use the divide-and-conquer algorithm to split the array into smaller parts and calculate their sum. Test your program with an array of 100 million integers.

Lesson 4. Use ThreadPoolExecutor to calculate the sum of elements in an array.

Create a ThreadPoolExecutor to sum the elements in an array using

multiple threads.

REFERENCES

- [1] Core Java: Fundamentals (2021) , Cay Horstmann (Oracle Press Java).
- [2] Head First Java: A Brain-Friendly Guide (2022), Kathy Sierra, O'Reilly Media.
- [3] Java OOP Done Right: Create object oriented code you can be proud of with modern Java Paperback (2019), Mr Alan Mellor, Mellor Books.
- [4] Murach's Java Programming (5th Edition) (2017), Joe Murach, Mike Murach & Associates.
- [5]. Java for Absolute Beginners Learn to Program the Fundamentals the Java 9+ Way.
- [6]. Modern Java Recipes: Simple Solutions to Difficult Problems in Java 8 and 9 (2017), by Ken Kousen, O'Reilly Media.
- [7] Effective Java (2018), Joshua Bloch, Addison-Wesley Professional.